
Fast Tree Diffusions for Probabilistic Tabular Regression via Flow Matching

Abstract

We extend Treeffuser, a score-based diffusion model for probabilistic tabular regression with gradient-boosted trees, in two directions motivated by fast sample-based probabilistic modelling. First, we evaluate conditional-mean residualization, EDM-style preconditioning, log-sigma time sampling, and noise-level features; after per-dataset tuning these raise CRPS skill score from 0.670 to 0.681, reduce 95% coverage error from 0.023 to 0.015, and cut sampling time $6.5\times$ over the published baseline. Second, we replace score matching with flow matching using Gaussian probability paths, use a closed-form conditional velocity-to-score mapping, and benchmark the resulting sampler family. With a VP path and deterministic Heun ODE at five steps, the flow-matching variant gives the best normalized CRPS among Treeffuser rows and is another $2.4\times$ faster than the improved score model, but gives back some of the score model’s coverage advantage. The common mechanism is better conditioning of the supervised regression problem seen by the tree ensemble. Against six per-dataset tuned probabilistic baselines (NGBoost, quantile-regression LightGBM, CatBoost uncertainty, iBUG, a deep ensemble, and CARD-style neural diffusion), the Treeffuser variants have the best aggregate normalized CRPS and coverage metrics under this protocol, while tuned non-diffusion baselines still win several individual datasets on raw CRPS. Stochastic ($\varepsilon > 0$) samplers do not Pareto-dominate the deterministic corner, so the empirical recommendation is the $\varepsilon = 0$ specialization when aggregate CRPS and latency are the target.

1 INTRODUCTION

Probabilistic regression on tabular data must combine the predictive strength of gradient-boosted trees with full predictive distributions usable for calibration, decision-making, and uncertainty quantification. Treeffuser [Beltrán-Velez et al., 2024] achieves this by training LightGBM [Ke et al., 2017] regressors to estimate the score of a noise-perturbed conditional density and integrating the corresponding reverse-time stochastic differential equation (SDE) [Song et al., 2021] at inference. The framework inherits the calibration properties of score-based diffusion while avoiding neural-network density estimators.

Our notion of tractability is computational rather than likelihood-exact: the model returns Monte Carlo samples, not closed-form marginal likelihoods. The tractable object is the inference procedure itself: the final FM row uses a five-step Heun ODE, Eq. (2) converts velocities to scores in closed form when stochastic sampling is desired, and training reduces to supervised tree regression.

This paper develops Treeffuser along two axes that interact with the tree-based score regressor in ways that differ from the neural-network case. *First*, on the score-training side, we evaluate four modifications motivated by the histogram-bin density structure of tree learners. *Second*, we replace score matching with flow matching using Gaussian probability paths [Lipman et al., 2023, Liu et al., 2023], which removes the indirect score-to-drift mapping that score-matching imposes on tree predictions and opens access to the stochastic-interpolant family [Albergo et al., 2023], parameterized by a non-negative stochasticity schedule $\varepsilon(t)$ that interpolates between the deterministic ODE sampler ($\varepsilon \equiv 0$) and the score-based reverse SDE.

Contributions. We make three contributions:

1. *Score-side modifications for tree-based diffusion.* An evaluation of conditional-mean residualization, EDM preconditioning, log- σ time sampling, and an explicit

log- σ feature in the histogram-split regime of LightGBM (§2.1).

2. *Flow matching with a closed-form velocity-to-score mapping.* A Gaussian-path FM trainer for Treeffuser, using Eq. (2) to recover the implied conditional score in closed form and to run stochastic samplers at any $\varepsilon(t) \geq 0$ (§2.2, Lemma 1).
3. *Regression-conditioning unification.* We identify regression conditioning for the tree ensemble—not the functional class itself—as the bottleneck behind both score- and flow-side gains, formalised through Propositions 3, 1, and 2 (§4, Appendix A).

Across ten UCI probabilistic-regression benchmarks, with one fold reserved for tuning and five folds held out for evaluation, the score-side modifications give the best aggregate calibration and CRPS skill score, while flow matching gives the best normalized CRPS and the fastest Treeffuser sampler. The shared lesson is that tree-based diffusion models depend strongly on whether the noising path, parameterization, training distribution, features, and sampler make the induced supervised problem well matched to histogram splits. Stochastic samplers ($\varepsilon > 0$) do not Pareto-dominate the deterministic ODE corner on any aggregate metric, so the empirical recommendation collapses the one-parameter family to $\varepsilon = 0$.

2 METHOD

2.1 SCORE-SIDE MODIFICATIONS

The published Treeffuser uses VESDE marginals with uniform t sampling, a noise-prediction target, and a feature layout $[y_t, X, t]$ given to LightGBM. We change four pieces, each motivated by tree-based regression rather than borrowed wholesale from neural diffusion. They act on distinct levers: response/input conditioning, allocation of training rows over noise levels, explicit noise-level features, and sampler efficiency.

Conditional-mean residualization. We fit a cross-validated LightGBM mean predictor $\hat{\mu}(x)$ on the training data and apply diffusion to $y - \hat{\mu}(x)$ rather than y directly. This converts a heteroscedastic conditional density into one whose mode is approximately zero and whose remaining variation is scale only, which empirically simplifies the score-regression target. We use a high-capacity configuration (no depth cap, 63 leaves, $n_{\text{est}}=300$, $\eta=0.05$) chosen by a separate sweep.

EDM preconditioning. Following Karras et al. [2022], we predict a denoised target $D_\theta(y_t, t)$ scaled to unit variance across noise levels and reconstruct the score $s(y_t, t) = (D_\theta - y_t)/\sigma(t)^2$. This stabilizes the regression target for

trees, which would otherwise need to extrapolate noise-prediction magnitudes across t .

Log-sigma time sampling. We draw $\log \sigma(t) \sim \mathcal{N}(-1.2, 1.2^2)$, clip to the SDE’s achievable range, and invert. Trees bin features by data density, so the distribution of training t values directly controls where the score model spends capacity. This is a stronger lever than loss reweighting [Hang et al., 2023], which scales loss within fixed bins.

Log-sigma noise feature. The feature vector for LightGBM becomes $[y_t, X, t, \log \sigma(t)]$, giving the regressor an explicit noise-level signal aligned with EDM scaling.

2.2 FLOW MATCHING WITH GAUSSIAN PATHS

A Gaussian flow path interpolates data y_0 and a standard-normal prior z via

$$y_t = \alpha(t) y_0 + \beta(t) z, \quad t \in [0, 1], \quad (1)$$

with boundary conditions $\alpha(0) = 1, \alpha(1) = 0, \beta(0) = 0, \beta(1) = 1$. We compare three paths: *linear* ($\alpha = 1 - t, \beta = t$), *trigonometric* ($\alpha = \cos(\pi t/2), \beta = \sin(\pi t/2)$), and a *VP* schedule with linear β_t giving $\alpha^2(t) = \exp(-T(t))$, $T(t) = \frac{1}{2}\beta_{\min}t + \frac{1}{4}(\beta_{\max} - \beta_{\min})t^2$. The training target is the conditional velocity $u_t = \alpha'(t)y_0 + \beta'(t)z$, regressed against the model’s $v_\theta(y_t, x, t)$.

Velocity-to-score formula. For any Gaussian path with Wronskian $W(t) = \alpha(t)\beta'(t) - \alpha'(t)\beta(t)$, the conditional score is recoverable in closed form:

$$\nabla \log p_t(y_t | x) = \frac{\alpha'(t)y_t - \alpha(t)v_\theta(y_t, x, t)}{W(t)\beta(t)}. \quad (2)$$

For linear FM this reduces to $-(y_t + (1 - t)v)/t$; for trig to $-y_t - (2/\pi) \cot(\pi t/2)v$.

Stochastic-interpolant family. Albergo et al. [2023] show that any non-negative $\varepsilon(t)$ defines a marginal-preserving reverse-time sampler at the population velocity/score:

$$dy_\tau = \left(-v_\theta + \frac{\varepsilon^2}{2}s_\theta\right) d\tau + \varepsilon dW_\tau, \quad (3)$$

where $\tau = 1 - t$ is reverse time and s_θ is the score from (2). The choice $\varepsilon \equiv 0$ recovers the deterministic probability-flow ODE; positive $\varepsilon(t)$ broadens samples, with exact marginal preservation only for the true velocity/score. We use the schedule $\varepsilon(t) = ct$, which vanishes at the data endpoint where the learned score’s $1/\beta(t)$ factor would otherwise dominate (see Proposition 2).

Why these choices suit trees. Three facts guide the implementation (derivations in Appendix A). First, Eq. (2) makes FM compatible with the same stochastic-interpolant samplers as score matching. Second, variance-preserving paths keep the scale of the tree input y_t nearly stationary; a linear path instead has $\text{Var}(y_t) \approx (1-t)^2 + t^2$ after standardization, which compresses the middle of the path and changes split geometry. Third, $\varepsilon = 0$ is a valid member of the sampler family, while positive ε multiplies approximate-score error by the endpoint-sensitive $1/\beta(t)$ factor.

2.3 IMPLEMENTATION NOTES

The flow-matching code reuses Treeffuser’s per-output-dimension LightGBM fitter and feature-construction pipeline; only the target and prior change. Sampling uses the same Heun integrator as the score path. The residualizer is unchanged across score and FM variants—a single conditional mean is used regardless of objective. At sample time, samples are inverse-transformed through the residualizer to recover the original y scale.

3 EXPERIMENTS

Datasets and protocol. Ten UCI benchmarks (Table 2) are evaluated with a nested tuning protocol. We split each dataset into six folds: fold 0 is used only for hyperparameter selection, and folds 1–5 are evaluation folds. For each dataset/model family, Optuna tunes on the fold-0 train/validation split; the selected configuration is then refit and evaluated on each held-out evaluation fold. Evaluation-fold training rows range from 256 on yacht to 38,108 on protein. For each test point we draw 200 samples via Heun integration; metrics are averaged over evaluation folds and, for headline summaries, over datasets.

Metrics. We report CRPS skill score $\text{CRPSS} = 1 - \text{CRPS}_{\text{model}}/\text{CRPS}_{\text{climatology}}$, where climatology is the empirical CDF of y_{train} ; mean rel-CRPS (per-dataset CRPS divided by the best variant on that dataset, then averaged); the Dawid–Sebastiani score [Dawid, 1984]; the fraction of evaluation folds at which a Kolmogorov–Smirnov test on the probability integral transform fails to reject uniformity at $\alpha = 0.05$ ("PIT KS $p > .05$ "); absolute coverage error at the 50/90/95% nominal interval levels [Gneiting and Raftery, 2007]; and mean sample-generation wall time per benchmark row. Timings are measured on the same Apple-arm64 workstation, exclude fitting, and include generating 200 samples per test point plus inverse transformations.

Variants compared. Three Treeffuser rows attribute contributions cleanly: (1) PUBLISHED—the original baseline with noise-prediction, uniform t , no residualization; (2) SCORE+—the baseline plus §2.1; (3) FM—VP flow

matching with residualization and a 5-step Heun ODE. Each row is tuned per dataset under the same fold-0 protocol. We also compare against six per-dataset tuned probabilistic baselines: NGBoost [Duan et al., 2020], quantile-regression LightGBM [Ke et al., 2017], CatBoost RMSE-with-uncertainty [Prokhorenkova et al., 2018], iBUG [Brophy and Lowd, 2022], a Gaussian deep ensemble [Lakshminarayanan et al., 2017], and CARD-style neural diffusion [Han et al., 2022].

3.1 HEADLINE RESULT

Table 1 reports cross-dataset means under the tuning/evaluation protocol. The score-side modifications lift CRPSS from 0.670 to 0.681, raise PIT-uniformity pass rate from 0.40 to 0.62, reduce 90/95% coverage error from 0.044/0.023 to 0.020/0.015, and cut Treeffuser sample time $6.5\times$. Flow matching is close on CRPSS, has the best normalized CRPS among all displayed rows, and is another $2.4\times$ faster than score+, for a $15.4\times$ speedup over the published sampler. The tradeoff is calibration: FM gives back part of score+’s 90/95% coverage advantage. Non-diffusion baselines sample faster still; Appendix H shows that tuned external baselines win some individual datasets on raw CRPS, while the Treeffuser rows lead on aggregate normalized CRPS and coverage. Paired Wilcoxon tests across the ten datasets show directional but not 0.05-level CRPS gains for score+ over published (one-sided $p = 0.080$) and FM over published ($p = 0.065$); FM versus score+ remains indistinguishable (two-sided $p = 0.922$).

3.2 PER-DATASET CRPSS

Table 2 shows the Treeffuser per-dataset CRPSS. The improved variants still dominate the small-to-mid regime, but the split is sharper after per-dataset tuning: score+ is best on concrete and diabetes, while FM is best on energy, kin8nm, naval, power, and yacht. The published baseline remains strongest on wine and on the two largest datasets. The resulting message is not that one recipe wins everywhere, but that the score-side modifications and the FM path expose complementary Pareto points: score+ is usually the calibrated choice, FM is the fast normalized-CRPS choice, and the published SDE can remain competitive when the full diffusion sampler is affordable.

3.3 STOCHASTICITY ABLATION

We sweep $\varepsilon \in \{0.25, 0.5, 1.0\}$ with schedule $\in \{\text{linear}, \sqrt{t}\}$ on the VP path, matched on residualizer (the C config used in the headline). Across all six SDE settings, no configuration Pareto-dominates the $\varepsilon = 0$ corner (Treeffuser-FM): CRPSS, |cE|@90, and |cE|@95 all degrade monotonically as ε grows, and sample time rises with

Table 1: Headline real-data results, mean over 10 UCI benchmarks with fold 0 used for tuning and folds 1–5 used for evaluation. rel-CRPS is the per-dataset CRPS divided by the best displayed variant on that dataset. $|cE|@q$ is absolute coverage error at the $q\%$ interval level.

Variant	CRPSS $\uparrow$	rel-CRPS $\downarrow$	DSS $\downarrow$	KS $p > .05 \uparrow$	$ cE @50 \downarrow$	$ cE @90 \downarrow$	$ cE @95 \downarrow$	samp. time
Treeffuser-published	0.670	1.242	1.358	0.40	0.117	0.044	0.023	221.35 s
Treeffuser-score ⁺ (ours)	0.681	1.131	−0.214	0.62	0.069	0.020	0.015	34.12 s
Treeffuser-FM (ours)	0.680	1.117	−0.218	0.46	0.064	0.028	0.024	14.41 s
QReg-LightGBM	0.668	1.459	0.016	0.34	0.066	0.041	0.044	0.30 s
CatBoost-unc.	0.638	1.409	0.272	0.28	0.093	0.078	0.066	0.01 s
NGBoost	0.601	1.550	1.957	0.16	0.119	0.114	0.101	0.04 s
Deep ensemble	0.654	1.519	0.090	0.26	0.090	0.032	0.018	0.03 s
iBUG	0.649	1.471	1.85×10^7	0.36	0.111	0.051	0.065	2.89 s
CARD-style diffusion	0.631	1.632	0.436	0.24	0.086	0.057	0.043	42.84 s

Table 2: Per-dataset CRPSS (higher is better) and post-split training rows.

Dataset	Train rows	Published	Score ⁺	FM
yacht	256	0.941	0.962	0.962
diabetes	368	0.201	0.241	0.226
energy	640	0.956	0.962	0.964
concrete	858	0.740	0.774	0.771
wine	5414	0.346	0.305	0.307
kin8nm	6826	0.555	0.617	0.631
power	7973	0.829	0.839	0.843
naval	9945	0.950	0.949	0.951
cali. housing	17200	0.687	0.676	0.674
protein	38108	0.494	0.480	0.472

the SDE solver overhead. This is consistent with Proposition 2: positive ε couples the SDE drift to a $1/\beta(t)$ -amplified approximate score, and the ct schedule used here suppresses but does not remove this possible error-amplification mechanism. The closest contender (linear schedule, $\varepsilon = 0.25$) matches $|cE|@95$ at 0.015 but is strictly worse on CRPSS, $|cE|@50$, and KS pass rate; its timing comes from the 25-step SDE sweep rather than the full-protein headline protocol (Appendix F.1). This aggregate result should not be read as stochasticity never helping conditionally: the high-IQR diagnostic in Appendix J shows a tail-coverage tradeoff on the largest datasets. The recommendation is therefore deterministic ODE for aggregate CRPS/latency, with stochasticity left as a conditional-calibration knob.

3.4 PATH ABLATION

Across the same ten datasets we compare linear, trigonometric, and VP paths with the same residualizer and ODE sampler. VP is the strongest on CRPS and tail coverage; trig matches VP on calibration but is mildly worse on CRPS; linear lags both on $|cE|@95$ (mean 0.029 vs. 0.015). The advantage tracks variance preservation (Proposition 1): VP and trig satisfy $\alpha^2 + \beta^2 = 1$, keeping y_t at roughly unit scale across t , which stabilizes the tree regressor’s feature space relative to the linear-path schedule.

3.5 NEGATIVE TRANSFERS FROM SCORE TO FM

Three score-side tricks were ported to FM and rejected by matched sweeps. Log-noise t sampling changes CRPS by $< 0.4\%$ relative to uniform- t ; min-SNR loss weighting either matches uniform ($\gamma = 5$) or hurts CRPS ($\gamma = 1$); and mean-scale residualization worsens CRPS by 8–11%. The FM headline therefore keeps the simpler recipe: uniform t , uniform loss weighting, and mean residualization.

4 DISCUSSION

Two findings deserve emphasis. First, the per-row attribution table separates score-side and FM contributions cleanly, so the FM result is not implicitly absorbing improvements that belong to the score track. Second, per-dataset tuning turns the headline into a tradeoff rather than a single-winner story: score⁺ is the strongest calibrated Treeffuser row, whereas FM is the strongest normalized-CRPS and latency row. The common mechanism behind both gains is regression conditioning for tree ensembles: EDM preconditioning stabilises the VE score path’s inputs and denoising residual, while VP-FM avoids much of the same conditioning problem through the path geometry itself. This is not a single target-scaling story; log- σ sampling controls where tree capacity is spent, the log- σ feature exposes the noise regime, and Heun ODE sampling is an inference-side efficiency gain. The large-dataset inversion argues against a blanket “SDE is worse” interpretation: per-IQR-bin diagnostics show that stochasticity can improve high-uncertainty tail coverage, while the aggregate metrics penalise the same runs for broader intervals, worse CRPS, and higher latency.

Limitations. The comparison is now stronger because all displayed families are tuned per dataset, but it is still finite-budget model selection: the Optuna spaces differ in dimensionality and computational cost. We include NGBoost, iBUG, CatBoost uncertainty, quantile-regression LightGBM, a deep ensemble, and a CARD-style neural diffusion baseline, but not distributional random forests [Ce-

vid et al., 2022]. Headline tables average five evaluation folds after one tuning fold, so per-dataset gaps, especially on the largest datasets, should be read as descriptive rather than as high-powered significance estimates. The remaining modelling limitation is conditional selection: choosing between score⁺, FM, and stochastic samplers should ultimately be driven by the user’s calibration and latency target, not by aggregate CRPS alone.

Future work. Two threads merit investigation: budget-normalized tuning across probabilistic families, and explicit selection criteria for stochastic-interpolant samplers when calibrated *conditional* (rather than marginal) coverage matters.

References

- Michael S. Albergo, Nicholas M. Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv:2303.08797*, 2023.
- Nicolas Beltrán-Velez, Alessandro Antonio Grant, Alexander Feder, Cinjon Resnick, Kyunghyun Cho, and David M. Blei. Treeffuser: Probabilistic predictions via conditional diffusions with gradient-boosted trees. In *Advances in Neural Information Processing Systems*, 2024.
- Jonathan Brophy and Daniel Lowd. Instance-based uncertainty estimation for gradient-boosted regression trees. In *Advances in Neural Information Processing Systems*, 2022.
- Domagoj Cevid, Loris Michel, Jeffrey Naef, Nicolai Meinshausen, and Peter Bühlmann. Distributional random forests: Heterogeneity adjustment and multivariate distributional regression. *Journal of Machine Learning Research*, 23(333):1–79, 2022.
- A. Philip Dawid. Statistical theory: The prequential approach. *Journal of the Royal Statistical Society. Series A*, 147(2):278–292, 1984.
- Tony Duan, Anand Avati, Daisy Yi Ding, Khanh K. Thai, Sanjay Basu, Andrew Y. Ng, and Alejandro Schuler. NG-Boost: Natural gradient boosting for probabilistic prediction. In *International Conference on Machine Learning*, 2020.
- Tilman Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.
- Xizewen Han, Huangjie Zheng, and Mingyuan Zhou. CARD: Classification and regression diffusion models. In *Advances in Neural Information Processing Systems*, 2022.
- Tiankai Hang, Shuyang Gu, Chen Li, Jianmin Bao, Dong Chen, Han Hu, Xin Geng, and Baining Guo. Efficient diffusion training via min-SNR weighting strategy. In *International Conference on Computer Vision*, 2023.
- Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. In *Advances in Neural Information Processing Systems*, 2022.
- Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 2017.
- Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, 2017.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.
- Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations*, 2023.
- Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, 2018.
- Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021.

A THEORETICAL DERIVATIONS AND DESIGN CONSEQUENCES

Lemma 1 (Wronskian velocity-to-score identity) *For a Gaussian path $y_t = \alpha(t)y_0 + \beta(t)z$ with conditional velocity $u_t = \alpha'(t)y_0 + \beta'(t)z$ and nonzero Wronskian $W(t) = \alpha(t)\beta'(t) - \alpha'(t)\beta(t)$, the population velocity $v(y_t, x, t) = \mathbb{E}[u_t | y_t, x, t]$ implies the conditional score*

$$\nabla \log p_t(y_t | x) = \frac{\alpha'(t)y_t - \alpha(t)v(y_t, x, t)}{W(t)\beta(t)}.$$

Proof. Solving the two linear equations for z gives

$$z = \frac{\alpha(t)u_t - \alpha'(t)y_t}{\alpha(t)\beta'(t) - \alpha'(t)\beta(t)} = \frac{\alpha(t)u_t - \alpha'(t)y_t}{W(t)}.$$

Because $p(y_t | y_0, x) = \mathcal{N}(\alpha y_0, \beta^2 I)$, the conditional score is $-z/\beta(t)$. Fisher’s identity gives $\nabla \log p_t(y_t | x) = \mathbb{E}[-z/\beta(t) | y_t, x, t]$. Substituting the expression for z and using that y_t is fixed under the conditional expectation yields the claim. Replacing v by v_θ gives Eq. (2). $\square$

At a fixed t with $\alpha(t) \neq 0$, the same identity inverts to $v(y_t, x, t) = \alpha'(t)y_t/\alpha(t) - W(t)\beta(t)\nabla \log p_t(y_t | x)/\alpha(t)$. Thus score and velocity have the same conditional structure as functions of y_t at that time; the practical differences studied here come from path-dependent scale variation across t , feature geometry, and sampler dynamics rather than an intrinsic fixed- t smoothness advantage.

Proposition 1 (Path-induced feature variance) *Let y_0, z be independent with $\text{Var}(y_0) = \text{Var}(z) = I$ after standardization, and let $y_t = \alpha(t)y_0 + \beta(t)z$. Then $\text{Var}(y_t) = \alpha(t)^2 + \beta(t)^2$. For the three paths used in this paper:*

- *Linear ($\alpha = 1 - t, \beta = t$): $\text{Var}(y_t) = (1 - t)^2 + t^2$, with minimum $1/2$ at $t = 1/2$.*
- *Trigonometric ($\alpha = \cos(\pi t/2), \beta = \sin(\pi t/2)$): $\text{Var}(y_t) = 1$ for all t .*
- *VP ($\alpha^2(t) = e^{-T(t)}, \beta^2(t) = 1 - e^{-T(t)}$): $\text{Var}(y_t) = 1$ for all t .*

Proof. Independence and unit variance give $\text{Var}(y_t) = \alpha^2 \text{Var}(y_0) + \beta^2 \text{Var}(z) = \alpha^2 + \beta^2$. Substituting the three path definitions yields the listed expressions; the trig and VP identities follow from $\cos^2 + \sin^2 = 1$ and from $\alpha^2 + \beta^2 = e^{-T} + (1 - e^{-T}) = 1$. $\square$

Consequence for tree features. Proposition 1 formalises why trig and VP paths keep the tree input near unit scale across the entire path, whereas the linear path compresses to half-variance at $t = 1/2$. For histogram-split trees this matters because the empirical distribution of y_t determines both candidate split locations and where leaf resolution is spent.

Residualization removes predictable location. If $\mu(x) = \mathbb{E}[y | x]$ and $r = y - \mu(x)$, then $\mathbb{E}[\|r\|^2 | x] = \text{tr Var}(y | x) \leq \mathbb{E}[\|y\|^2 | x]$. Diffusion or FM on residuals therefore removes a deterministic location component that would otherwise be present at every noise level. The inequality also explains the large-dataset caveat: a fixed residualizer can become a bottleneck once the inner score/velocity model has enough data to represent the mean directly.

Proposition 2 (Endpoint score-error amplification) *Let $v_\theta(y_t, x, t) = v(y_t, x, t) + \delta(y_t, x, t)$ be an approximate velocity with pointwise error δ , and let $\hat{s}_\theta, s$ denote the implied and true scores from Lemma 1. Then*

$$\hat{s}_\theta(y_t, x, t) - s(y_t, x, t) = -\frac{\alpha(t)}{W(t)\beta(t)} \delta(y_t, x, t),$$

so the pointwise score-error gain is $|\alpha(t)/(W(t)\beta(t))|$. For all three paths considered here $W(t) \rightarrow \alpha(0)\beta'(0) > 0$ is bounded near $t = 0$ while $\beta(t) \rightarrow 0$, so the gain diverges at the data endpoint.

Proof. Eq. (2) is linear in v , so

$$\hat{s}_\theta - s = \frac{\alpha'(t)y_t - \alpha(t)(v + \delta) - (\alpha'(t)y_t - \alpha(t)v)}{W(t)\beta(t)} = -\frac{\alpha(t)\delta}{W(t)\beta(t)}.$$

$\square$

Endpoint amplification. Proposition 2 therefore predicts that any stochasticity schedule with $\varepsilon(t) \not\rightarrow 0$ as $t \rightarrow 0$ injects high-gain approximate score into the SDE drift; the schedule $\varepsilon(t) = ct$ used in §2.2 and the deterministic $\varepsilon \equiv 0$ corner are the two natural responses.

Proposition 3 (Population equivalence on the VE path) *For a VE path $y_t = y_0 + \beta(t)z$ with $\beta'(t) > 0$, training a normalized velocity model ν_θ against $u_t/\beta'(t) = z$ implies the same population score as VE noise prediction and VE denoising/EDM score reconstruction:*

$$s_t(y_t, x) = -\frac{\mathbb{E}[z \mid y_t, x, t]}{\beta(t)} = \frac{\mathbb{E}[y_0 \mid y_t, x, t] - y_t}{\beta(t)^2}.$$

Proof. For VE, $\alpha = 1$, $\alpha' = 0$, $u_t = \beta'(t)z$, and $W = \beta'(t)$. Lemma 1 gives $s_t = -\mathbb{E}[u_t \mid y_t, x, t]/(\beta(t)\beta'(t)) = -\mathbb{E}[z \mid y_t, x, t]/\beta(t)$. Since $y_t = y_0 + \beta(t)z$, $\mathbb{E}[y_0 \mid y_t, x, t] = y_t - \beta(t)\mathbb{E}[z \mid y_t, x, t]$, which gives the denoising form. Noise prediction estimates $-\mathbb{E}[z \mid y_t, x, t]$ and EDM reconstructs the same population denoiser after its skip/output/input preconditioning. $\square$

Proposition 3 is a population identity, not a claim that the finite LightGBM problems are identical. EDM fits the preconditioned denoising residual $F = (y_0 - c_{\text{skip}}y_t)/c_{\text{out}}$ and reconstructs $D_\theta = c_{\text{skip}}y_t + c_{\text{out}}F_\theta$, whereas normalized VE-FM would fit z directly. They imply the same score at the optimum but present different targets and feature scalings to LightGBM. This is the useful interpretation of the score⁺-FM tie: both improve the conditioning of the regression problem seen by the tree ensemble, EDM by preconditioning the VE score objective and VP-FM by choosing a better-conditioned probability path.

Table 3: Regression-conditioning view of the main design choices. Here y_0 and z are standardized. “Conditioning issue” refers to the supervised problem fit by LightGBM, not to the population score identity.

Variant	Regressed object	Feature scale	Conditioning issue / fix
VE noise prediction	$-z$	$y_0 + \sigma z$ grows with σ	Unit target, but heteroscedastic input and $1/\sigma$ score amplification.
VE EDM score ⁺	$(y_0 - c_{\text{skip}}y_t)/c_{\text{out}}$	$c_{\text{in}}y_t$ is near unit scale	Input and denoising residual are explicitly preconditioned.
Raw VE-FM	$\beta'(t)z$	$y_0 + \beta z$ grows with β	Velocity target inherits the schedule derivative.
Normalized VE-FM	z	$y_0 + \beta z$ unless also scaled	Population-equivalent to noise prediction after known rescaling.
VP-FM	$\alpha'y_0 + \beta'z$	$\alpha^2 + \beta^2 \approx 1$	Path keeps tree features near the standardized data scale.

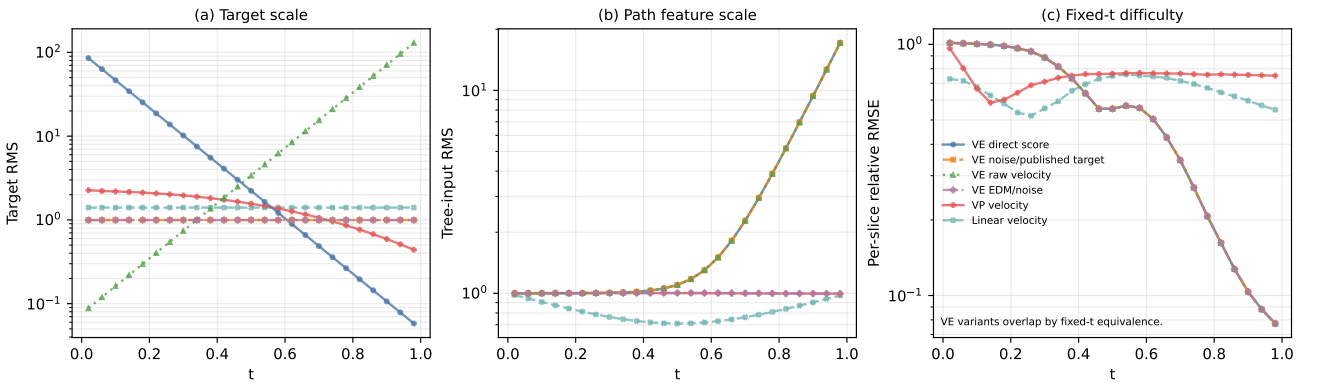


Figure 1: Toy preconditioning diagnostic on a one-dimensional heteroscedastic mixture, generated by `pixi run toy-geometry`. Panel (a) shows that direct VE score prediction and raw VE-FM have target scales that vary by orders of magnitude across t , whereas the published-style VE noise target, EDM/noise prediction, and linear velocity have unit target scale. Panel (b) separates target scaling from input geometry: the published-style VE target is stable but the tree input $y_t = y_0 + \sigma z$ still expands strongly, while EDM rescales the input and VP-FM keeps the path feature near standardized scale. Panel (c) fits a fresh LightGBM at each fixed t ; the VE curves overlap up to numerical noise, confirming that the score/velocity distinction is not intrinsic fixed- t smoothness but cross- t conditioning.

B DATASETS AND PROTOCOL DETAILS

Sources. The ten UCI benchmarks follow the canonical Treeffuser protocol [Beltrán-Velez et al., 2024]: yacht, concrete, energy, wine, kin8nm, naval, power_plant, protein from the UCI repository; california_housing from scikit-learn’s fetch_california_housing; and diabetes from scikit-learn’s load_diabetes. All raw row counts are used before the fixed 90/10 train/test split (Table 2 reports post-split training rows); the only preprocessing is StandardScaler standardisation of features and target, fit on the training split and applied to the corresponding test split. wine is the union of wine-quality-red and wine-quality-white with a binary color indicator.

Seed policy. Seeds 0–2 (headline) and 0–9 (ablations) drive three independent streams: data_seed (offset 0) for the train/test split; model_seed (offset 10,000) for the LightGBM regressors; sampler_seed (offset 20,000) for the stochastic-interpolant sampler. Variance reduction across seeds therefore reflects only sampling-and-split variability, not boosting initialisation.

Common training settings. Unless noted, all Treeffuser variants use $n_{\text{estimators}} = 3000$, early stopping after 50 rounds, learning rate 0.1, $n_{\text{repeats}} = 30$ (each training point is reused with 30 fresh t samples), and the per-output-dimension LightGBM fitter. Inference draws 200 samples per test point and integrates with the Heun method.

Sampler details. TREEFFUSER-PUBLISHED uses the VESDE reverse-time SDE with the Euler–Maruyama sampler at $n_{\text{steps}} = 50$, matching the published code path. TREEFFUSER-SCORE⁺ uses the Heun probability-flow ODE at $n_{\text{steps}} = 25$ steps under the EDM-style parametrization with $\sigma_{\min} = 0.01$, $\sigma_{\max} = 20$. TREEFFUSER-FM uses the Heun ODE at $n_{\text{steps}} = 5$ on the VP path with $\beta_{\min} = 0.1$, $\beta_{\max} = 20$.

C SCORE-SIDE MODIFICATIONS: ABLATION CROSS-REFERENCES

The four score-side modifications of §2.1 were validated by smoke and synthetic-core sweeps before being combined in the SCORE⁺ variant. The residualizer-C configuration that the headline uses was selected by the residualizer sweep summarised in Appendix D. Log-sigma time sampling, the log-sigma noise feature, and EDM preconditioning were locked in by the log_sigma_sweep, synthetic_core, and pf_ode_sweep development runs. We treat those runs as model-selection evidence rather than as separately reported ablations; the quantitative claim in the paper is the combined SCORE⁺ row in Table 1, where all four choices are evaluated together on the full ten-dataset protocol.

D RESIDUALIZER CONFIGURATION

D.1 OFF VS. MEAN RESIDUALIZATION (FM)

We isolate the contribution of mean residualization on the FM side at matched configuration (VP path, uniform t , uniform loss weighting, residualizer-C extra parameters) using ten datasets and ten seeds.

Table 4: FM with mean residualization vs. no residualization (off). Paired differences (mean – off, 100 paired observations); negative is better for all metrics shown.

Sampler	ΔCRPS	$\Delta q\text{-MACE}$	$\Delta\text{KS stat}$	$\Delta \text{cE} @90$	paired t on ΔCRPS
ODE @ 5 steps	−0.23	−0.022	−0.034	−0.032	−4.30
SDE @ 25 steps	−0.05	+0.001	+0.003	−0.001	−2.91

The ODE-5 advantage is large and broadly significant across calibration metrics (81/100 paired wins on $q\text{-MACE}$ and 81/100 on $|\text{cE}|@90$). At SDE-25 the gap on calibration metrics is null (paired $t < 1$); the aggregate CRPS gain is driven by two real-data outliers (yacht +54% relative, concrete +15%) while six of ten datasets actually favour the unresidualized baseline. The mechanism is consistent with the few-step ODE being unable to absorb the conditional mean itself when the velocity net has ≤ 5 Heun updates; with 25 SDE updates the velocity model recovers calibration on its own. Cost: mean residualization adds ≈ 12 s of fit time on these sizes ($\sim 8\times$ vs. no residualization), so the trade is most attractive in low-step regimes.

D.2 RESIDUALIZER VARIANTS A–E (FM SIDE)

Five residualizer configurations were swept on a four-dataset, three-seed sub-suite (california_housing, diabetes, energy, protein-5k subsample) under VP-FM-ODE at 5 steps; see Table 5.

Table 5: FM residualizer ablation. “Conf. leaves” is the LightGBM `num_leaves`; “ $n_{\text{est}}^{(r)}$ ” is the residualizer’s `n_estimators`; “ES” is early-stopping rounds in the residualizer (off means no ES).

Variant	Description	Leaves	$n_{\text{est}}^{(r)}$	ES	CRPSS $\uparrow$	q -MACE $\downarrow$	$ \text{cE} @95 \downarrow$	fit s
A	current-baseline	31	100	off	0.521	0.028	0.017	1.82
B	regularised	31	100	30	0.509	0.042	0.018	2.78
C	high capacity	63	300	off	0.523	0.039	0.011	8.03
D	ES moderate	63	500	30	0.516	0.033	0.021	5.91
E	ES + high capacity	63	2000	30	0.522	0.034	0.010	9.81

Configurations C and E sit on the calibration–latency Pareto front. C is the headline choice (best CRPSS, $1.2\times$ faster than E); E edges C on tail coverage ($|\text{cE}|@95$ 0.010 vs. 0.011). The full-protein run of §3.1 shows E reclaiming the lead on protein (CRPSS 0.476 vs. C’s 0.449), reinforcing the case that the optimal residualizer is dataset-dependent.

D.3 SCALE-AWARE RESIDUALIZATION

The `mean_scale` residualizer additionally divides residuals by a cross-validated conditional-std estimate $\hat{\sigma}(x)$. On the ten-dataset / ten-seed protocol:

Table 6: Mean vs. mean-scale residualization on FM at matched configuration. Higher CRPSS is better; lower CRPS, DSS, q -MACE are better.

Variant	CRPSS $\uparrow$	CRPS $\downarrow$	DSS $\downarrow$	q -MACE $\downarrow$
VP-FM-ODE, mean	0.519	0.652	+0.314	0.031
VP-FM-ODE, mean-scale	0.489	0.724	+0.665	0.038
VP-FM-SDE, mean	0.517	0.657	+0.380	0.040
VP-FM-SDE, mean-scale	0.475	0.750	+0.676	0.049

Scale-aware residualization is uniformly worse: +11% CRPS on the ODE path, +14% on the SDE path, with DSS more than doubling. The per- x scale estimator is itself noisy at $n = 1000$; dividing by it injects more variance into the velocity target than it removes. `mean` is therefore the only residualizer carried into the headline.

E TIME SAMPLING AND LOSS WEIGHTING

E.1 TIME SAMPLING

Table 7: FM time-sampling sweep, 10 datasets $\times$ 3 seeds, matched residualizer-C, VP path. “uniform” is the headline choice with an endpoint anchor at $t = 1$ at probability 0.05.

Variant	CRPSS $\uparrow$	DSS $\downarrow$	q -MACE $\downarrow$	KS $p > .05 \uparrow$	samp s
VP-FM-ODE uniform	0.6564	−0.154	0.040	0.47	3.63
VP-FM-ODE uniform anchor16	0.6560	−0.171	0.040	0.53	3.84
VP-FM-ODE logbeta	0.6556	−0.175	0.035	0.77	3.74
VP-FM-ODE logSNR	0.6538	−0.135	0.042	0.40	3.71
score ⁺ uniform- t	0.6556	−0.210	0.038	0.50	6.95
score ⁺ log- σt	0.6565	−0.169	0.038	0.70	6.58

For FM, log-beta and log-SNR sampling do not move CRPS by more than 0.4% relative to uniform with endpoint anchor; the smoother density shape gains +5% on KS pass rate and -13% on q -MACE but the effect is within seed variance for CRPSS. The headline keeps uniform- t for parsimony; log-beta is a viable substitute when KS uniformity is the priority. For the score model, log-sigma sampling clearly helps PIT uniformity (KS pass $0.50 \rightarrow 0.70$) at the cost of slightly worse DSS; we keep log-sigma in the SCORE⁺ headline because the KS gain is the more interpretable calibration win.

E.2 LOSS WEIGHTING (MIN-SNR- γ)

Table 8: Min-SNR- γ loss weighting on FM, 10 datasets $\times$ 3 seeds. $\gamma = 5$ collapses to uniform on most of the support; $\gamma = 1$ amplifies the late-time loss most aggressively.

Variant	CRPSS $\uparrow$	CRPS $\downarrow$	DSS $\downarrow$	q -MACE $\downarrow$	KS $p > .05 \uparrow$
VP-FM-ODE uniform- w	0.6564	4.097	-0.159	0.040	0.47
VP-FM-ODE min-SNR-5	0.6564	4.119	-0.167	0.037	0.60
VP-FM-ODE min-SNR-1	0.6516	4.223	-0.104	0.044	0.37
VP-FM-SDE uniform- w	0.6540	4.114	-0.080	0.051	0.30
VP-FM-SDE min-SNR-5	0.6538	4.138	-0.085	0.050	0.30
VP-FM-SDE min-SNR-1	0.6485	4.239	-0.046	0.057	0.23

Min-SNR- γ is a sharpness-versus-tail trade-off. $\gamma = 1$ hurts CRPS by 0.8–1.3% across both samplers (Wilcoxon $p \leq 0.003$ for both); $\gamma = 5$ is statistically indistinguishable from uniform weighting on CRPS, but improves PIT KS pass rate on the ODE path. We keep uniform weighting in the FM headline.

F STOCHASTICITY (VELOCITY-NOISE) SWEEPS

F.1 STOCHASTICITY STRENGTH

We sweep velocity stochasticity $\varepsilon \in \{0.25, 0.50, 1.0\}$ on the VP path under both linear and $\sqrt{t}$ schedules with residualizer-C; results in Table 9 are averages over 10 datasets $\times$ 3 seeds at 25 SDE steps.

Table 9: Stochasticity sweep. “lo025” means $\varepsilon = 0.25$ etc. The $\varepsilon = 0$ corner is the headline FM (Heun ODE @ 5 steps).

Variant	CRPSS $\uparrow$	CRPS $\downarrow$	cE @90 $\downarrow$	cE @95 $\downarrow$	samp s
$\varepsilon = 0$ headline (ODE)	0.669	–	0.024	0.017	8.7
$\varepsilon = 0.25$ linear, res-C	0.656	4.099	0.028	0.015	5.7
$\varepsilon = 0.50$ linear, res-C	0.655	4.104	0.036	0.017	5.9
$\varepsilon = 1.00$ linear, res-C	0.654	4.114	0.047	0.022	5.9
$\varepsilon = 0.50$ sqrt, res-C	0.655	4.101	0.035	0.017	5.9
$\varepsilon = 1.00$ sqrt, res-C	0.655	4.107	0.043	0.020	5.9
$\varepsilon = 1.00$ linear, res-E	0.651	4.128	0.048	0.024	9.8

No SDE setting Pareto-dominates $\varepsilon = 0$. The closest contender ($\varepsilon = 0.25$ linear) matches |cE|@95 (0.015 vs. 0.017) but is strictly worse on CRPSS, |cE|@90, and pays 25 SDE steps vs. 5 ODE steps. Sample-time differences in the table reflect the SDE entries being measured at 25 steps without the full-protein dataset cost; the headline ODE row reflects the cross-dataset mean over the full-protein protocol.

F.2 SCHEDULE SHAPE

The schedule $\varepsilon(t) = ct$ (linear) is the headline form. The $\sqrt{t}$ alternative was tested for parity at $\varepsilon \in \{0.5, 1.0\}$ and produces statistically indistinguishable CRPSS (Table 9). Both schedules vanish at the data endpoint, which prevents the $1/\beta(t)$ blow-up in the velocity-to-score conversion (Eq. (2)); neither shape helps once ε is small.

G PATH CHOICE

Table 10: Flow-path ablation: linear vs. trigonometric vs. variance-preserving, 8 datasets $\times$ 3 seeds, matched residualizer and ODE sampler. Lower is better for all columns; bold is best per column.

Path	CRPS $\downarrow$	q -MACE $\downarrow$	cE @95 $\downarrow$	samp s
Linear	4.320	0.026	0.025	1.30
Trigonometric	4.317	0.030	0.017	1.08
Variance-preserving (VP)	4.306	0.029	0.016	1.22

VP edges trig on CRPS and tail coverage; both Pareto-dominate linear on |cE|@95. The advantage tracks variance preservation: VP and trig satisfy $\alpha^2 + \beta^2 \approx 1$, keeping y_t at roughly unit scale across t and stabilising the tree regressor’s feature space. The headline uses VP.

H PROBABILISTIC-FAMILY BASELINES

We compare the headline Treeffuser variants against six external probabilistic regressors (Table 11). Hyperparameters for every displayed family are selected per dataset on the same fold-0 tuning split, then evaluated on folds 1–5. The CARD row uses a compact benchmark adapter with the same two-stage conditional-mean plus conditional-diffusion structure as CARD [Han et al., 2022].

Table 11: Probabilistic baselines under the tuning/evaluation protocol. rel-CRPS is normalized by the best displayed variant on each dataset. Sample-time excludes fitting.

Model	CRPSS $\uparrow$	rel-CRPS $\downarrow$	CRPS $\downarrow$	q -MACE $\downarrow$	cE @95 $\downarrow$	samp s
Treeffuser-score ⁺ (ours)	0.681	1.131	4.057	0.028	0.015	34.12
Treeffuser-FM-fast (ours)	0.680	1.117	4.121	0.034	0.024	14.41
Treeffuser-published	0.670	1.242	4.279	0.044	0.023	221.35
Quantile-regression LightGBM	0.668	1.459	4.026	0.033	0.044	0.30
Deep ensemble	0.654	1.519	4.008	0.050	0.018	0.03
iBUG (XGBoost)	0.649	1.471	4.193	0.069	0.065	2.89
CatBoost (uncertainty)	0.638	1.409	4.045	0.045	0.066	0.01
CARD-style diffusion	0.631	1.632	4.838	0.051	0.043	42.84
NGBoost (Gaussian)	0.601	1.550	4.299	0.052	0.101	0.04

Under this per-dataset tuning protocol, the Treeffuser variants lead on aggregate CRPSS, rel-CRPS, and coverage metrics. The external baselines are still important: deep ensembles have the best mean raw CRPS, though this scale-sensitive average is less informative about cross-dataset dominance than CRPSS or rel-CRPS, and the per-dataset winners include deep ensembles on *diabetes* and *kin8nm*, CARD on *naval*, and quantile-regression LightGBM on *protein*. Their aggregate rel-CRPS remains worse, indicating that no single tuned external family transfers as evenly across the benchmark suite as the Treeffuser variants.

I EXTENDED PER-DATASET RESULTS

Table 12 reports CRPS, |cE|@90, and sample-generation time on each of the ten benchmarks for the three headline Treeffuser variants, averaged over the five evaluation folds. The “best margin” column gives the gap between the winning variant and the second-best Treeffuser variant in raw CRPS. After per-dataset tuning, the margins are tighter: the main result is no longer a few dramatic wins, but a more balanced Pareto surface between published SDE sampling, score-side conditioning, and FM speed.

FM is the fastest Treeffuser sampler on 9/10 datasets and is especially valuable at scale: on *protein* the speedup is $19.3\times$ over PUBLISHED and $2.0\times$ over SCORE⁺; on *california_housing* the corresponding speedups are $31.6\times$ and $2.7\times$. On |cE|@90, however, SCORE⁺ leads on 6/10 datasets and FM on 3/10, which is why Table 1 separates normalized-CRPS and calibration conclusions. The per-dataset CRPS ranking reproduces the headline pattern: FM has the best average normalized CRPS, score⁺ is the more reliable calibration row, and the published SDE remains best on *wine*, *california housing*, and *protein*.

Table 12: Extended per-dataset Treeffuser results, ordered roughly by evaluation-fold training size. “best margin” is winner-over-second-best in raw CRPS; the bold cell in each block is the per-metric winner on that row. Numbers come from `benchmarks/results/tuning/eval/*.jsonl`.

Dataset	best margin	CRPS ↓			cE @90 ↓			sample s		
		Pub.	Score ⁺	FM	Pub.	Score ⁺	FM	Pub.	Score ⁺	FM
yacht	FM +0.4%	0.438	0.284	0.283	0.092	0.025	0.081	0.3	0.3	0.1
diabetes	Sco ⁺ +1.9%	35.682	34.025	34.660	0.079	0.029	0.038	2.4	0.5	0.2
energy	FM +5.2%	0.252	0.214	0.204	0.081	0.034	0.022	2.9	0.8	0.6
concrete	Sco ⁺ +1.5%	2.457	2.126	2.158	0.034	0.025	0.023	3.0	4.8	0.8
wine	Pub +5.9%	0.300	0.319	0.318	0.037	0.006	0.016	100.4	37.0	11.9
kin8nm	FM +4.0%	0.067	0.058	0.055	0.012	0.004	0.023	22.0	20.2	9.2
power	FM +2.4%	1.671	1.572	1.535	0.013	0.004	0.006	14.6	6.7	7.6
naval	FM +3.2%	0.000218	0.000220	0.000211	0.078	0.065	0.061	207.4	91.2	29.5
cali. housing	Pub +3.6%	0.197	0.204	0.206	0.010	0.005	0.007	605.9	52.6	19.2
protein	Pub +2.7%	1.722	1.769	1.796	0.005	0.005	0.007	1254.7	127.0	64.9

J CONDITIONAL TAIL CALIBRATION DIAGNOSTIC

The large-dataset inversion in Table 2 admits two plausible mechanisms: the fixed residualizer may be suboptimal at high n , and stochastic samplers may improve conditional tail calibration in high-uncertainty regions. Table 13 reports direct 95% coverage on `california_housing` and `protein`, including the highest predicted-IQR bin.

Table 13: High-uncertainty tail diagnostic on the two largest datasets. Cov@95 is marginal empirical coverage; Top-IQR Cov@95 is empirical coverage in the highest predicted-IQR quintile; both target 0.95. IQR-MACE is mean |cE|@95 across predicted-IQR quintiles, so lower is better. SDE-linear uses a linear stochasticity schedule; SDE- $\sqrt{t}$ uses the square-root schedule. All rows use the full training and test splits, averaged over the same three seeds as the headline run.

Dataset	Variant	CRPS	Cov@95	IQR-MACE	Top-IQR Cov@95
cali. housing	Published SDE	0.207	0.958	0.015	0.931
	FM ODE-C	0.214	0.947	0.021	0.900
	FM ODE-E	0.213	0.947	0.021	0.900
	FM SDE-C linear	0.215	0.969	0.023	0.939
	FM SDE-C $\sqrt{t}$	0.215	0.967	0.022	0.938
	FM SDE-E linear	0.213	0.971	0.023	0.944
	FM SDE-E $\sqrt{t}$	0.213	0.969	0.023	0.944
protein	Published SDE	1.787	0.944	0.018	0.925
	FM ODE-C	1.860	0.933	0.025	0.923
	FM ODE-E	1.771	0.949	0.008	0.943
	FM SDE-C linear	1.859	0.959	0.011	0.960
	FM SDE-C $\sqrt{t}$	1.856	0.956	0.012	0.954
	FM SDE-E linear	1.779	0.974	0.024	0.973
	FM SDE-E $\sqrt{t}$	1.776	0.971	0.021	0.971

The diagnostic supports a tradeoff rather than a one-sided conclusion. On `california_housing`, the deterministic FM rows under-cover the high-IQR bin (0.900), while adding FM stochasticity lifts this coverage to 0.938–0.944; the price is marginal over-coverage (0.967–0.971) and no gain over the published SDE on CRPS or IQR-MACE. On `protein`, residualizer-E with the deterministic ODE is already close to the nominal target and remains best on CRPS and IQR-MACE, whereas the residualizer-E SDE rows over-cover both marginally and in the high-IQR bin. Residualizer-C stochasticity is directionally useful for coverage, especially with the square-root schedule, but gives up CRPS. Thus the safest interpretation is not that SDE samplers are dominated, but that their benefit is conditional and must be selected against conditional-calibration metrics rather than headline CRPS alone.

K REPRODUCIBILITY

All sweeps in this paper are driven by YAML configs under `benchmarks/configs/`. Each result file is a JSON Lines document recording the full hyperparameter set, random seeds, the git commit hash of the model code, a hash of the Treeffuser source tree, and every reported metric on a per-dataset/per-fold row. The headline tuning runs persist their selected configurations under `benchmarks/results/tuning/best_params/` and their five-fold evaluation rows under `benchmarks/results/tuning/eval/`. The configs that produced the appendix tables are listed alongside in Table 14.

Table 14: Provenance: appendix tables and the config / result file each was produced from.

Table	Config	Result file
4	fm_off_vs_mean_sweep.yaml	fm_off_vs_mean_sweep__*.jsonl
3	analytic summary	-
5	residualizer_fm_sweep.yaml	residualizer_fm_sweep__*.jsonl
6	fm_mean_scale_sweep.yaml	fm_mean_scale_sweep__*.jsonl
7	fm_t_sampling_sweep.yaml	fm_t_sampling_sweep__*.jsonl
8	fm_loss_weighting_sweep.yaml	fm_loss_weighting_sweep__*.jsonl
9	sde_lower_stoch_sweep.yaml, stochasticity_schedule_sweep.yaml	sde_lower_stoch_sweep__*.jsonl, stochasticity_schedule_sweep__*.jsonl
10	flow_path_sweep.yaml	flow_path_sweep__*.jsonl
11, 2, 12, 1	tuning_manifest.yaml	benchmarks/results/tuning/eval/*.jsonl
13	paper_real_data_v2.yaml, tail_calibration_full.yaml	paper_real_data_v2_full.jsonl, tail_calibration_full.jsonl